YNOV CAMPUS / JOUR 1

Jour 1 Construire des bases solides

Rappels, conception relationnelle, contraintes et SQL sûr. Apprenez avec des exemples commentés, puis exercez-vous dans le TP01.

PostgreSQL 18 | 2e année de bachelor | Édition pédagogique progressive

Pourquoi une application a besoin d’une base de données

Une boutique en ligne doit conserver les produits, les clients et les achats après la fermeture de l’application. Une base de données est un ensemble organisé de faits enregistrés. Un système de gestion de base de données, ou SGBD, est le logiciel qui stocke ces faits et contrôle leur consultation et leur modification. Dans ce cours, PostgreSQL est notre SGBD et CampusMarket notre boutique d’exemple.

Base de données : ensemble organisé de données enregistrées.

Suivez le raisonnement

1. L’application pose une question, comme « Quels produits coûtent moins de EUR 20 ? »
2. Le SGBD traite la demande et renvoie les données correspondantes.
3. L’application peut afficher le résultat sans stocker une copie distincte de chaque fait.

Une table contient des lignes et des colonnes

Une base de données relationnelle organise les faits en tables. Une ligne décrit une instance, par exemple un produit. Une colonne décrit une propriété, comme le nom ou le prix du produit. La table ci-dessous est un extrait de campus.products. Lisez une ligne horizontalement pour décrire un produit ; lisez une colonne verticalement pour comparer la même propriété entre produits.

Résultat / données d’exemple

product_id	name	unit_price
1	Keyboard	50.00
2	Mouse	25.00
3	Notebook	10.00

Une ligne représente un produit. C’est le sens, ou la granularité, d’une ligne.

SQL, PostgreSQL et le client ont des fonctions différentes

SQL signifie Structured Query Language, ou langage de requête structuré. C’est le langage utilisé pour décrire les opérations sur les bases de données. PostgreSQL comprend SQL et gère les données enregistrées. Un client est le programme par lequel nous envoyons du SQL à PostgreSQL. Vous pouvez utiliser psql dans un terminal ou DBeaver avec une interface graphique. Les deux peuvent envoyer la même instruction SQL.

Résultat / données d’exemple

Nom	Nature	Rôle dans ce cours
SQL	Un langage	Écrire des requêtes
PostgreSQL 18	Un SGBD/serveur	Stocker et traiter les données
psql / DBeaver	Clients de base de données	Envoyer du SQL et voir les résultats
Docker Compose	Outil d’environnement	Démarrer le serveur du cours
Pilote de base de données	Bibliothèque applicative	Connecter le code au serveur

Vous n’avez pas besoin d’une bibliothèque de programmation pour écrire du SQL dans psql ou DBeaver.

Suivre une requête de votre ordinateur au serveur

Le serveur est le processus PostgreSQL en cours d’exécution qui gère une base de données. Une connexion est une conversation entre un client et ce serveur. Le client envoie une instruction SQL ; le serveur renvoie des lignes ou une erreur. Deux onglets SQL peuvent partager une connexion ; un prochain TP sur la concurrence ouvrira donc explicitement deux connexions distinctes.

Sens des mots-clés

Client — Programme qui envoie la requête.

Serveur — Processus qui reçoit et exécute le SQL.

Connexion — Session avec une base de données et un rôle particuliers.

Jeu de résultats — Lignes renvoyées par une requête.

Suivre une requête de votre ordinateur au serveur / explication visuelle

Le serveur est le processus PostgreSQL en cours d’exécution qui gère une base de données. Une connexion est une conversation entre un client et ce serveur. Le client envoie une instruction SQL ; le serveur renvoie des lignes ou une erreur. Deux onglets SQL peuvent partager une connexion ; un prochain TP sur la concurrence ouvrira donc explicitement deux connexions distinctes.



Le SQL va vers le serveur. Les lignes de résultat ou les erreurs reviennent au client.

Préparer l’environnement du cours

Nos exemples de commandes utilisent Bash, un interpréteur de commandes de terminal. Utilisez Terminal sous macOS/Linux ou Git Bash/WSL sous Windows. Docker exécute PostgreSQL dans un conteneur, un environnement d’exécution isolé. Compose lit compose.yaml et démarre ce conteneur, donnant à toute la classe la même version de base de données. DBeaver est facultatif. Si vous utilisez PowerShell, prenez les commandes de configuration sur une ligne du README du kit au lieu des continuations Bash par barre oblique inverse.

Suivez le raisonnement

1. Installez et démarrez Docker Desktop, ou utilisez le moteur Docker fourni par le campus.
2. Extrayez l’archive des supports étudiants. Repérez le dossier sql contenant compose.yaml.
3. Ouvrez un terminal dans ce dossier. Un terminal est une application permettant de saisir des commandes textuelles.
4. DBeaver est facultatif ; psql est déjà disponible dans le conteneur.

Connaître les fichiers avant de les exécuter

Un script est un fichier texte contenant des commandes exécutées dans l’ordre. Le kit de cours sépare la structure des tables des données d’exemple pour que chacun ait un objectif clair. Le script de schéma est un script de réinitialisation : il supprime les schémas du cours avant de les recréer. Utilisez-le uniquement dans la base campusmarket dédiée au cours, et enregistrez vos propres fichiers SQL en dehors de la base de données.

Résultat / données d’exemple

Fichier	Objectif
compose.yaml	Démarrer PostgreSQL 18 en local
student/00_schema.sql	Créer les tables et contraintes ; réinitialiser les schémas du cours
student/01_seed.sql	Insérer le jeu de données du cours à six produits
student/02_checks.sql	Vérifier l’état initial attendu
student/TP01_starter.sql	Un fichier pour écrire votre propre travail

Démarrer le serveur depuis le bon dossier

Les commandes suivantes sont des commandes de terminal, pas du SQL. La première démarre la base en arrière-plan et attend son contrôle de bon fonctionnement. La seconde affiche l’état du service. Le service s’appelle db dans compose.yaml. Laissez Docker fonctionner pendant votre travail. Si la commande ne trouve pas le fichier de configuration, vérifiez que votre terminal est dans le dossier sql.

TERMINAL / COMMANDES CLIENT

```
docker compose up -d --wait
docker compose ps
```

Sens des mots-clés

up — Créer et démarrer le service configuré.

-d — L’exécuter en arrière-plan.

--wait — Attendre que le service soit prêt.

ps — Afficher l’état du service.

Ne saisissez pas le symbole d’invite du terminal avant la commande.

Charger la structure, puis les données d’exemple

Une base de données ne contient initialement aucune table du cours. Exécutez ces commandes de terminal dans l’ordre indiqué. psql est le client SQL du service db. L’option -f lit un fichier de script. Le chemin /course désigne l’emplacement du dossier étudiant du kit dans le conteneur. Une barre oblique inverse en fin de ligne du terminal prolonge la commande sur la ligne suivante.

TERMINAL / COMMANDES CLIENT

```
docker compose exec -T db psql -U course_admin \
  -d campusmarket -f /course/00_schema.sql
docker compose exec -T db psql -U course_admin \
  -d campusmarket -f /course/01_seed.sql
docker compose exec -T db psql -U course_admin \
  -d campusmarket -f /course/02_checks.sql
```

Sens des mots-clés

-U — Se connecter avec le rôle indiqué.

-d — Choisir une base de données ; ici campusmarket.

-f — Lire le SQL depuis le fichier indiqué.

-T — Ne pas allouer de terminal interactif pour ce script.

Ouvrir une session psql interactive

Une session interactive permet de saisir du SQL et d’examiner immédiatement chaque résultat. Exécutez la commande de terminal ci-dessous. Lorsque l’invite campusmarket apparaît, saisissez les deux instructions SQL affichées après le commentaire. Le point-virgule termine une instruction SQL. Les commandes commençant par une barre oblique inverse sont des instructions psql et non du SQL ; utilisez \q pour quitter psql.

TERMINAL / COMMANDES CLIENT

```
docker compose exec db psql -U course_admin \
  -d campusmarket

-- Après l’apparition de l’invite psql, saisissez du SQL :
SET TIME ZONE 'UTC';
SELECT current_database(), current_user;
```

Suivez le raisonnement

1. Exécutez d’abord uniquement la commande docker dans le terminal.

2. Saisissez ensuite SET et SELECT dans la session psql.

3. Vous devez obtenir la base campusmarket et le rôle course_admin.

Se connecter avec DBeaver au lieu de psql

DBeaver est un autre client du même serveur en cours d’exécution. Créez une connexion PostgreSQL et remplissez les champs ci-dessous. L’hôte identifie l’ordinateur ; le port identifie le service de base de données sur cet ordinateur. Ouvrez un éditeur SQL associé à cette connexion. Testez-la avec SELECT current_database(); avant d’exécuter un script de réinitialisation.

Résultat / données d’exemple

Champ de connexion	Valeur pour le cours
Hôte	127.0.0.1 ou localhost
Port	5433
Base de données	campusmarket
Nom d’utilisateur	course_admin
Mot de passe	classroom_only_change_me

Se connecter avec DBeaver au lieu de psql / raisonnement pas à pas

Suivez ces étapes en parallèle de l’exemple SQL précédent. Prévoyez ce que chaque étape reçoit et produit avant de l’exécuter. L’essentiel n’est pas seulement de savoir si l’instruction s’exécute, mais pourquoi ces opérations répondent à la question posée avec les données fournies.

Suivez le raisonnement

1. Créez une nouvelle connexion de base de données et sélectionnez PostgreSQL.
2. Saisissez ces valeurs, testez la connexion, puis ouvrez un éditeur SQL.
3. Utilisez une nouvelle connexion physique lorsqu’un TP demande explicitement une session B.

Un schéma donne son nom complet à une table

Une base de données PostgreSQL peut contenir plusieurs schémas. Un schéma est un espace de noms : il regroupe des objets nommés, comme les tables. Dans campus.products, campus est le schéma et products la table. Les véritables données du cours sont dans campus ; les essais jetables vont dans work. Écrire le nom complet évite de dépendre d’un réglage invisible de recherche des schémas.

Résultat / données d’exemple

Niveau	Exemple	Objectif
Base de données	campusmarket	La base de données sélectionnée par la connexion
Schéma	campus	Espace de noms contenant les tables du jeu de données de référence
Table	products	Lignes décrivant les produits
Nom qualifié	campus.products	Nom du schéma suivi du nom de la table

Base de données : campusmarket. Schémas : campus et work. Table : campus.products.

Un schéma donne son nom complet à une table / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

campus.products — Nom du schéma, point, nom de la table.

search_path — Schémas parcourus dans l’ordre pour un nom non qualifié.

Base de données : campusmarket. Schémas : campus et work. Table : campus.products.

Vérifier l’état initial et diagnostiquer les erreurs

Avant d’étudier la syntaxe des requêtes, vérifiez que chacun accède aux mêmes données. Un comptage donne le nombre de lignes. Les vérifications ci-dessous doivent renvoyer 6 et 10. Si une relation n’existe pas, la table nommée n’a pas été trouvée avec cette connexion. Vérifiez la base de données, l’orthographe du schéma et l’ordre de configuration avant de modifier le SQL au hasard.

SQL / lire l’instruction une clause à la fois

```sql
SELECT count(*) AS students
FROM campus.students;

SELECT count(*) AS orders
FROM campus.orders;
```

Résultat / données d’exemple

Vérification	Attendu
students	6
orders	10

Sens des mots-clés

AS — Donner un nom lisible à une colonne de sortie.

count(*) — Compter chaque ligne ; étudié en détail plus tard aujourd’hui.

Une erreur est un retour d’information : lisez son message et son emplacement, puis testez une explication.

SELECT et FROM : demander des colonnes précises

Une instruction SELECT lit des données et renvoie un jeu de résultats. SELECT nomme les colonnes de sortie. FROM identifie la table d’entrée. La requête utile la plus simple combine ces deux clauses. Une clause est une partie nommée d’une instruction SQL. La base de données évalue la demande ; elle ne vous oblige pas à écrire une boucle sur les lignes. SELECT * demande toutes les colonnes d’entrée.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name, unit_price
FROM campus.products;
```

Sens des mots-clés

SELECT — Choisir les expressions à afficher.

FROM — Choisir la table d’entrée.

, — Séparer les expressions sélectionnées.

; — Terminer l’instruction.

SELECT * — Renvoyer toutes les colonnes de l’entrée ; des colonnes explicites sont plus claires pour des rapports stables.

Lire le résultat du premier SELECT

La requête renvoie deux colonnes pour chaque produit, car aucune condition ne limite les lignes. Elle ne modifie pas la table enregistrée. Sélectionner moins de colonnes ne supprime pas les autres colonnes de la base. Cet affichage présente les lignes du jeu de données de référence pour l’explication ; une requête sans ORDER BY ne garantit aucun ordre d’affichage particulier.

Résultat / données d’exemple

name	unit_price
Keyboard	50.00
Mouse	25.00
Notebook	10.00
Bottle	20.00
USB Hub	30.00
Sticker Pack	5.00

Le résultat est une nouvelle représentation des données, pas une nouvelle table enregistrée.

Utiliser AS pour nommer une expression de sortie

Un alias est un nom temporaire utilisé dans une requête ou son résultat. AS attribue un alias sans renommer la colonne enregistrée. Ici, price_eur indique aux lecteurs ce que signifie la valeur numérique. Un alias de colonne est utile lorsqu'une expression a un nom long, qu'un calcul nécessite une étiquette ou qu'un rapport attend un contrat de sortie particulier.

SQL / lire l'instruction une clause à la fois

```sql
SELECT product_id,
       name,
       unit_price AS price_eur
FROM campus.products;
```

Sens des mots-clés

AS price_eur — Nommer cette colonne de résultat price_eur.

Alias — Un nom au niveau de la requête, pas une opération ALTER TABLE.

La colonne d'origine s'appelle toujours unit_price.

ORDER BY rend l’ordre d’affichage explicite

Les lignes d’une table n’ont aucun ordre de présentation garanti. ORDER BY trie le résultat selon une ou plusieurs expressions. ASC signifie croissant et DESC décroissant. Les clés de tri sont évaluées de gauche à droite. Une dernière clé unique rend l’ordre déterministe : les ex æquo sont départagés de façon constante au lieu de laisser ce choix indéfini.

SQL / lire l’instruction une clause à la fois

```
SELECT product_id, name, unit_price
FROM campus.products
ORDER BY unit_price DESC, product_id ASC;
```

Résultat / données d’exemple

D’abord	Puis	Enfin
Keyboard 50	USB Hub 30, Mouse 25, Bottle 20, Notebook 10	Sticker Pack 5

Sens des mots-clés

ORDER BY — Trier les lignes renvoyées.

DESC — Du plus grand au plus petit.

ASC — Du plus petit au plus grand ; sens par défaut.

LIMIT et OFFSET sélectionnent une page après le tri

LIMIT limite le nombre de lignes de résultat renvoyées. OFFSET saute un nombre de lignes avant de renvoyer la page. Pour demander les deux produits les plus chers, triez par prix décroissant, puis limitez à deux. Sans ordre pertinent, « les deux premiers » n’est pas une demande métier fiable. Plus tard, nous comparerons ce style de pagination à la pagination par clé.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name, unit_price
FROM campus.products
ORDER BY unit_price DESC, product_id
LIMIT 2;
```

Résultat / données d’exemple

name	unit_price
Keyboard	50.00
USB Hub	30.00

Sens des mots-clés

LIMIT 2 — Renvoyer au plus deux lignes.

OFFSET 2 — Si ajouté, sauter les deux premières lignes triées.

Les littéraux sont des valeurs écrites directement dans le SQL

Un littéral est une valeur incluse directement dans une instruction. Les littéraux textuels utilisent des apostrophes, les littéraux numériques n’en utilisent pas, et TRUE et FALSE représentent des valeurs booléennes. Un booléen est une valeur décrivant la vérité. Les guillemets doubles identifient un nom ; ils ne sont pas les délimiteurs normaux du texte. PostgreSQL autorise aussi les expressions SELECT sans table FROM.

SQL / lire l’instruction une clause à la fois

```sql
SELECT 'Keyboard' AS label,
       50.00 AS price,
       TRUE AS available;
```

Résultat / données d’exemple

label	price	available
Keyboard	50.00	true

Sens des mots-clés

'Keyboard' — Un littéral textuel.

50.00 — Un littéral numérique.

TRUE — Un littéral booléen.

"Name" — Un identifiant entre guillemets, comme un nom de colonne sensible à la casse.

Un type de données définit les valeurs qui ont un sens

Un type de données décrit les valeurs qu’une colonne peut stocker et les opérations possibles sur celles-ci. Un prix doit permettre des calculs décimaux exacts ; un nom de produit doit permettre des opérations textuelles. Le choix des types aide aussi la base à refuser les entrées incompatibles. NUMERIC(12,2) autorise douze chiffres au total, dont deux après le séparateur décimal ; il n’interdit pas à lui seul un prix négatif.

Résultat / données d’exemple

Type	Signification	Exemple
integer / bigint	Nombres entiers ; bigint a une plage plus étendue	quantity 3
numeric(12,2)	Valeurs décimales exactes	price 25.50
text	Chaînes de caractères	Keyboard
boolean	Vrai, faux ou inconnu via NULL	active true
date	Date du calendrier	2026-01-05
timestamptz	Un instant affiché selon un fuseau horaire	2026-01-05 09:00+00

Les expressions calculent une valeur pour chaque ligne

Une expression combine des valeurs, des opérateurs ou des fonctions pour produire une autre valeur. La multiplication utilise *. Cette requête calcule la valeur actuelle du stock au catalogue pour chaque produit : prix actuel multiplié par stock actuel. Le calcul ne modifie aucune des colonnes enregistrées. Cet indicateur n’est pas le chiffre d’affaires des ventes, car celui-ci utilisera plus tard le prix historique de chaque ligne de commande vendue.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name,
       unit_price * stock AS catalog_value
FROM campus.products
ORDER BY product_id
LIMIT 3;
```

Résultat / données d’exemple

name	catalog_value
Keyboard	1000.00
Mouse	750.00
Notebook	500.00

Sens des mots-clés

* — Multiplier les valeurs de part et d’autre.

Expression — Calcul produisant une valeur.

AS catalog_value — Nommer la sortie calculée.

CAST convertit une valeur vers un autre type de données

CAST demande à la base d’interpréter une valeur selon un type indiqué. Le type compte en arithmétique : la division entière de PostgreSQL élimine la partie fractionnaire. Convertir un opérande en numeric produit une division décimale. Une conversion doit être valide ; un texte comme « cheap » ne peut pas devenir un prix numérique. N’utilisez pas une conversion pour remplacer la vérification d’entrées peu fiables.

SQL / lire l’instruction une clause à la fois

```sql
SELECT 5 / 2 AS integer_result,
       CAST(5 AS numeric) / 2 AS decimal_result;
```

Résultat / données d’exemple

integer_result	decimal_result
2	2.5

Sens des mots-clés

CAST(value AS type) — Convertir vers le type nommé.

/ — Diviser ; les types des opérandes influent sur le résultat.

value::type — Raccourci PostgreSQL pour le même type de conversion.

WHERE conserve les lignes dont la condition est vraie

WHERE filtre les lignes d’entrée selon une condition. Une condition compare des valeurs et vaut vrai, faux ou inconnu. PostgreSQL conserve une ligne uniquement si la condition WHERE est vraie. La comparaison >= signifie supérieur ou égal à. Dans l’exemple, le filtrage précède le tri final ; la sortie contient donc uniquement les produits admissibles, triés par prix.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name, unit_price
FROM campus.products
WHERE unit_price >= 25
ORDER BY unit_price DESC, product_id;
```

Sens des mots-clés

WHERE — Conserver les lignes satisfaisant une condition.

>= — Supérieur ou égal à.

Condition / prédicat — Expression dont le résultat est une valeur de vérité.

Examiner ce que WHERE a écarté

Trois produits coûtent au moins EUR 25 : Keyboard, USB Hub et Mouse. Les trois autres ne sont pas supprimés ; ils n’apparaissent simplement pas dans ce résultat. Remplacez la condition >= 25 par > 25 et Mouse disparaît, car son prix vaut exactement 25. Les valeurs limites comptent lorsqu’on traduit une règle métier en comparaison.

Résultat / données d’exemple

name	unit_price
Keyboard	50.00
USB Hub	30.00
Mouse	25.00

Sens des mots-clés

= — Égal à.

<> — Différent de.

< / <= — Inférieur à / inférieur ou égal à.

> / >= — Supérieur à / supérieur ou égal à.

AND, OR et NOT combinent les conditions

AND exige que les deux conditions soient vraies. OR exige qu’au moins une soit vraie. NOT inverse une condition de vérité, avec un comportement inconnu particulier pour NULL, abordé sous peu. Les parenthèses rendent le regroupement voulu explicite. Cet exemple demande les produits actifs qui sont assez chers ou ont un stock faible ; un produit inactif doit toujours être exclu.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name, unit_price, stock
FROM campus.products
WHERE active
  AND (unit_price >= 20 OR stock < 15)
ORDER BY product_id;
```

Résultat / données d’exemple

name	unit_price	stock
Keyboard	50.00	20
Mouse	25.00	30
Bottle	20.00	40

Sens des mots-clés

AND — Les deux côtés doivent être vérifiés.

OR — Au moins un côté doit être vérifié.

NOT active — Sélectionner les valeurs active fausses ; NULL reste inconnu.

IN énumère des valeurs ; BETWEEN inclut les deux bornes

IN teste l’appartenance à une liste de valeurs. BETWEEN exprime une borne inférieure et une borne supérieure incluses. Pour les prix, BETWEEN 10 AND 25 équivaut à price >= 10 AND price <= 25. Le AND de BETWEEN appartient à cette expression d’intervalle. Ces formes rendent les filtres courants lisibles, mais il faut toujours décider si les bornes font partie de la règle métier.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name, unit_price
FROM campus.products
WHERE unit_price BETWEEN 10 AND 25
ORDER BY unit_price, product_id;
```

Résultat / données d’exemple

name	unit_price
Notebook	10.00
Bottle	20.00
Mouse	25.00

Sens des mots-clés

IN (2, 4) — Correspondre à l’une des valeurs listées, par exemple category_id.

BETWEEN 10 AND 25 — Inclure 10 et 25 ainsi que les valeurs entre eux.

LIKE recherche un motif textuel

LIKE compare du texte à un motif. Le caractère générique % correspond à toute suite de caractères, y compris aucune ; _ correspond à exactement un caractère. Un caractère générique est un substitut dans le motif, pas dans la structure SQL. PostgreSQL propose ILIKE, une forme insensible à la casse. La correspondance dépend des paramètres de comparaison du texte de la base ; testez donc les textes non anglais et accentués lorsque cela compte.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name
FROM campus.products
WHERE name LIKE '%book%'
ORDER BY product_id;
```

Résultat / données d’exemple

name
Notebook

Sens des mots-clés

LIKE — Rechercher un motif textuel.

% — Correspondre à zéro ou plusieurs caractères.

_ — Correspondre à un caractère.

ILIKE — Recherche de motif insensible à la casse dans PostgreSQL.

DISTINCT élimine les lignes de sortie en double

SELECT conserve normalement les lignes de résultat en double. DISTINCT élimine les doublons sur l’ensemble de la ligne sélectionnée. Comme plusieurs produits appartiennent à la même catégorie, sélectionner category_id répète les numéros de catégorie. Ajouter DISTINCT renvoie une fois chaque catégorie représentée. Cela ne corrige pas une jointure logiquement erronée et ne modifie pas les doublons de la table enregistrée.

SQL / lire l’instruction une clause à la fois

```
SELECT DISTINCT category_id
FROM campus.products
ORDER BY category_id;
```

Résultat / données d’exemple

category_id
2
3
4
5

Sens des mots-clés

DISTINCT — Conserver un exemplaire de chaque ligne sélectionnée distincte.

Ligne sélectionnée complète — Avec deux colonnes, l’unicité s’évalue sur la paire.

NULL signifie une information manquante ou inconnue

NULL est un marqueur d’information absente ou inconnue ; ce n’est ni zéro ni une chaîne vide. La catégorie racine All n’a pas de parent, donc parent_id vaut NULL. L’égalité ordinaire ne peut pas décider si deux valeurs inconnues sont égales. Utilisez IS NULL pour savoir si une valeur manque et IS NOT NULL pour savoir si elle est présente.

SQL / lire l’instruction une clause à la fois

```
SELECT category_id, name
FROM campus.categories
WHERE parent_id IS NULL
ORDER BY category_id;
```

Résultat / données d’exemple

category_id	name
1	All

Sens des mots-clés

NULL — Aucune valeur connue n’est présente.

IS NULL — Tester la présence du marqueur de valeur manquante.

IS NOT NULL — Tester qu’une valeur est présente.

Pourquoi une comparaison avec NULL peut échouer au filtre WHERE

SQL possède un troisième résultat de vérité : inconnu. Une comparaison ordinaire avec NULL produit généralement inconnu. WHERE ne conserve que vrai ; faux et inconnu sont donc exclus. Cela explique pourquoi parent_id = NULL ne trouve pas la catégorie racine. Pour les comparaisons traitant volontairement les valeurs manquantes comme comparables, PostgreSQL propose aussi IS DISTINCT FROM et IS NOT DISTINCT FROM.

Résultat / données d’exemple

Expression	Valeur de vérité	Conservé par WHERE ?
5 = 5	true	oui
5 = 7	false	non
NULL = 5	inconnu	non
NULL = NULL	inconnu	non
NULL IS NULL	true	oui
FALSE AND NULL	false	non

Ne remplacez pas une valeur manquante par zéro sauf si zéro a le sens métier voulu.

Pourquoi une comparaison avec NULL peut échouer au filtre WHERE / résultat commenté

SQL possède un troisième résultat de vérité : inconnu. Une comparaison ordinaire avec NULL produit généralement inconnu. WHERE ne conserve que vrai ; faux et inconnu sont donc exclus. Cela explique pourquoi parent_id = NULL ne trouve pas la catégorie racine. Pour les comparaisons traitant volontairement les valeurs manquantes comme comparables, PostgreSQL propose aussi IS DISTINCT FROM et IS NOT DISTINCT FROM.

Résultat / données d’exemple

Expression	Valeur de vérité	Conservé par WHERE ?
TRUE OR NULL	true	oui

Ne remplacez pas une valeur manquante par zéro sauf si zéro a le sens métier voulu.

COALESCE choisit une valeur de repli

COALESCE renvoie le premier argument non NULL. Il sert à définir un libellé d’affichage ou une valeur par défaut choisie délibérément. Ici, un identifiant de parent manquant est affiché comme 0 pour faciliter la lecture. Cela ne modifie pas le parent_id enregistré. Zéro est une convention d’affichage dans cet exemple, pas une nouvelle catégorie parente ni une signification universelle des données manquantes.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name,
       COALESCE(parent_id, 0) AS parent_display
FROM campus.categories
ORDER BY category_id;
```

Résultat / données d’exemple

name	parent_display
All	0
Technology	1
Lifestyle	1
Accessories	2
Stationery	3

Sens des mots-clés

COALESCE(a, b) — Renvoyer a s’il est présent ; sinon b.

Types compatibles — Les arguments doivent se résoudre en un type commun utilisable.

CASE transforme des conditions en une valeur calculée

CASE choisit une sortie selon des conditions. WHEN introduit une condition, THEN donne son résultat, ELSE une valeur de repli et END ferme l’expression. Les conditions sont testées dans l’ordre et la première vraie l’emporte. Une expression CASE crée une valeur dans SELECT ; ce n’est ni une boucle distincte ni une commande mettant la table à jour.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name,
  CASE WHEN unit_price >= 30 THEN 'premium'
       ELSE 'standard' END AS price_band
FROM campus.products
ORDER BY product_id
LIMIT 3;
```

Résultat / données d’exemple

name	price_band
Keyboard	premium
Mouse	standard
Notebook	standard

CASE transforme des conditions en une valeur calculée / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

CASE — Commencer la sélection conditionnelle d’une valeur.

WHEN ... THEN — Condition et valeur choisie.

ELSE — Valeur de repli si aucune condition n’est vraie.

END — Fermer cette expression CASE.

Les fonctions intégrées transforment une valeur

Une fonction prend des arguments et renvoie une valeur. PostgreSQL inclut des fonctions pour le texte, les nombres et les dates. TRIM retire les espaces initiaux et finaux ; LOWER convertit le texte en minuscules selon les règles textuelles. Les parenthèses contiennent les arguments. Lorsque les fonctions sont imbriquées, évaluez d’abord l’intérieur : retirez les espaces, puis passez le résultat en minuscules. SELECT ne modifie pas les données d’origine enregistrées.

SQL / lire l’instruction une clause à la fois

```sql
SELECT trim('  SQL  ') AS trimmed,
       lower(trim('  SQL  ')) AS normalized;
```

Résultat / données d’exemple

trimmed	normalized
SQL	sql

Sens des mots-clés

Function(argument) — Appliquer une opération nommée à une valeur.

trim(text) — Retirer par défaut les espaces qui entourent le texte.

lower(text) — Produire du texte en minuscules.

Dates et horodatages répondent à des questions différentes

Une date identifie un jour du calendrier. Un timestamp without time zone décrit une date et une heure locales sans choisir un instant à l’échelle mondiale. Le type PostgreSQL timestamptz identifie un instant et l’affiche selon le fuseau horaire de la session. Le décalage +00 de nos exemples signifie UTC. Nous utilisons systématiquement UTC afin qu’un filtre de date ait le même sens sur tous les ordinateurs du cours.

SQL / lire l’instruction une clause à la fois

```sql
SET TIME ZONE 'UTC';
SELECT DATE '2026-01-05' AS day,
       TIMESTAMPTZ '2026-01-05 09:00+00' AS instant;
```

Résultat / données d’exemple

day	instant
2026-01-05	2026-01-05 09:00:00+00

Sens des mots-clés

Littéral DATE — Valeur typée de date du calendrier.

Littéral TIMESTAMPTZ — Instant typé avec un décalage horaire.

UTC — Convention commune de fuseau horaire utilisée dans ce kit.

Utiliser un intervalle semi-ouvert pour un mois

La table orders contient une ligne par commande et un horodatage nommé ordered_at. Pour sélectionner les commandes de janvier, incluez le 1er janvier et excluez le 1er février. C’est un intervalle semi-ouvert : le début appartient à l’intervalle, la fin n’y appartient pas. Cela évite de deviner le dernier horaire possible du 31 janvier et s’articule proprement avec le mois suivant.

SQL / lire l’instruction une clause à la fois

```sql
SELECT order_id, ordered_at
FROM campus.orders
WHERE ordered_at >= TIMESTAMPTZ '2026-01-01+00'
  AND ordered_at <  TIMESTAMPTZ '2026-02-01+00'
ORDER BY ordered_at, order_id;
```

Résultat / données d’exemple

order_id	ordered_at UTC
101	2026-01-05 09:00
102	2026-01-05 09:00
103	2026-01-20 12:00

Sens des mots-clés

>= start — Inclure l’instant de début.

< next_start — Exclure l’instant de début de la période suivante.

À vous de jouer : choisir des produits actifs abordables

CampusMarket a besoin d’une courte liste pour une promotion de produits actifs. Renvoyez name et unit_price pour les produits actifs coûtant au moins EUR 20. Affichez les plus chers en premier et utilisez product_id pour départager les prix ex æquo. Écrivez la requête avant de regarder la diapositive suivante. Il suffit de SELECT, FROM, WHERE, AND et ORDER BY.

Suivez le raisonnement

1. Identifiez les deux colonnes de sortie demandées.
2. Écrivez les deux conditions : actif et prix d’au moins 20.
3. Triez par prix décroissant, puis par identifiant de produit croissant.

Vérifiez que le produit inactif USB Hub est exclu même si son prix est assez élevé.

Corrigé expliqué : combiner sélection, filtrage et tri

La liste SELECT définit les deux colonnes de sortie. FROM identifie la table source. WHERE combine deux exigences avec AND ; un produit inactif ne peut donc pas être retenu. ORDER BY place le prix le plus élevé en premier et départage les ex æquo par l’identifiant. Trois lignes restent. L’exécution doit donner exactement la table ci-dessous si les données initiales n’ont pas été modifiées.

SQL / lire l’instruction une clause à la fois

```sql
SELECT name, unit_price
FROM campus.products
WHERE active AND unit_price >= 20
ORDER BY unit_price DESC, product_id;
```

Résultat / données d’exemple

name	unit_price
Keyboard	50.00
Mouse	25.00
Bottle	20.00

Concevoir une table autour d’un seul type de fait

Avant de créer une table, indiquez ce que représente une ligne. Une proposition de produit est un produit suggéré pour le catalogue, pas une commande ni un paiement. Donnez une colonne à chaque propriété et choisissez des types adaptés à leur sens. Cette décision rend les contraintes et requêtes ultérieures compréhensibles. Mélanger des types de faits sans rapport dans une table crée des lignes confuses et des informations en double.

Résultat / données d’exemple

Propriété	Type choisi	Raison
product_id	integer	Un identifiant de produit
sku	text	Un code métier de catalogue
name	text	Un nom d’affichage
unit_price	numeric(12,2)	Un prix décimal exact
stock	integer	Une quantité entière

DDL signifie Data Definition Language, ou langage de définition des données : des instructions SQL qui définissent les objets de la base de données.

CREATE TABLE déclare les colonnes et leurs règles

CREATE TABLE ajoute une définition de table à un schéma. Entre parenthèses, chaque colonne est suivie de son type et de contraintes facultatives. Une contrainte est une règle que la base impose lors des changements de données. Le modèle ci-dessous décrit la grammaire ; les espaces réservés entre chevrons sont des explications à remplacer avant l’exécution. Nous définirons ensuite les règles individuelles avant d’exécuter une véritable instruction.

MODÈLE DE SYNTAXE / remplacer les espaces réservés

```
CREATE TABLE schema_name.table_name (
  column_name data_type constraint,
  another_column data_type
);
```

Sens des mots-clés

CREATE TABLE — Définir une nouvelle table.

Définition de colonne — Nom, type et règles facultatives.

Contrainte — Règle vérifiée par le SGBD.

, — Séparer les définitions de colonnes ou de tables.

Les clés identifient les lignes et relient les faits

Une clé candidate est un ensemble minimal de colonnes identifiant une ligne de façon unique. La clé primaire est la clé candidate choisie comme identifiant principal. Une clé étrangère exige qu’une valeur référence une clé parente autorisée. Une clé composée utilise plusieurs colonnes ensemble, comme order_id et product_id pour identifier une ligne de produit dans une commande.

Résultat / données d’exemple

Règle	Signification concrète
PRIMARY KEY (product_id)	Chaque produit a un identifiant unique et non NULL
UNIQUE (sku)	Les codes de catalogue ne peuvent pas se répéter
FOREIGN KEY student_id	Une commande doit référencer un étudiant existant
PRIMARY KEY (order_id, product_id)	Un produit apparaît une seule fois par clé de ligne de commande

Un nom est souvent une mauvaise clé primaire, car deux personnes peuvent le partager ou il peut changer.

NOT NULL, CHECK et UNIQUE résolvent des problèmes différents

NOT NULL exige la présence d’une valeur. CHECK exige qu’une condition booléenne ne soit pas fausse. UNIQUE empêche les valeurs de clé répétées selon ses règles relatives à NULL. Un CHECK sur price >= 0 n’exige pas à lui seul un prix, car une comparaison avec NULL vaut inconnu et non faux. Combinez NOT NULL et CHECK lorsqu’une valeur doit exister et respecter un intervalle.

Résultat / données d’exemple

Contrainte	Accepte	Refuse
NOT NULL	Une valeur présente	NULL
CHECK (stock >= 0)	0, 3 et NULL sauf interdiction distincte	-1
UNIQUE (sku)	Des codes de catalogue différents	Un code non NULL répété
PRIMARY KEY	Des identifiants uniques et présents	Des identifiants NULL ou répétés

Avec UNIQUE par défaut dans PostgreSQL, les NULL sont distincts ; NULLS NOT DISTINCT est une autre possibilité explicite.

Créer une table d’exercice complète avec contraintes

Voici une définition complète et exécutable de la table d’exercice du TP01. Le schéma work a été créé par le script de configuration. Chaque règle correspond à une exigence : tout produit a un identifiant, les codes de catalogue sont uniques, les noms sont présents, les prix sont positifs et le stock n’est pas négatif. Exécutez-la une fois dans un schéma work neuf ; CREATE TABLE signale une erreur si ce nom existe déjà.

SQL / lire l’instruction une clause à la fois

```sql
CREATE TABLE work.product_proposal (
  product_id integer PRIMARY KEY,
  sku text NOT NULL UNIQUE,
  name text NOT NULL,
  unit_price numeric(12,2) NOT NULL
    CHECK (unit_price > 0),
  stock integer NOT NULL CHECK (stock >= 0)
);
```

Suivez le raisonnement

1. Lisez d’abord le nom et le type de chaque colonne.

2. Lisez ensuite chaque contrainte comme une règle métier.

3. Le point-virgule termine l’instruction CREATE TABLE complète.

INSERT INTO ajoute de nouvelles lignes

INSERT INTO nomme la table cible et les colonnes à remplir. VALUES fournit les valeurs correspondantes dans le même ordre. Les valeurs textuelles utilisent des apostrophes. RETURNING demande à PostgreSQL de renvoyer la ligne insérée pour examiner ce qui a été enregistré. Préciser la liste des colonnes rend la correspondance visible et évite de dépendre de l’ordre physique des colonnes de la table.

SQL / lire l’instruction une clause à la fois

```sql
INSERT INTO work.product_proposal
  (product_id, sku, name, unit_price, stock)
VALUES (900, 'P-900', 'Desk Pad', 12.50, 8)
RETURNING product_id, name, unit_price, stock;
```

Sens des mots-clés

INSERT INTO — Choisir la table cible.

Liste des colonnes — Préciser la destination de chaque valeur fournie.

VALUES — Fournir une ou plusieurs lignes.

RETURNING — Afficher les valeurs des lignes qui viennent de changer.

INSERT INTO ajoute de nouvelles lignes / résultat commenté

INSERT INTO nomme la table cible et les colonnes à remplir. VALUES fournit les valeurs correspondantes dans le même ordre. Les valeurs textuelles utilisent des apostrophes. RETURNING demande à PostgreSQL de renvoyer la ligne insérée pour examiner ce qui a été enregistré. Préciser la liste des colonnes rend la correspondance visible et évite de dépendre de l’ordre physique des colonnes de la table.

Résultat / données d’exemple

product_id	name	unit_price	stock
900	Desk Pad	12.50	8

Une modification refusée indique la règle qui a échoué

Tenter d’insérer une autre ligne avec product_id 900 viole la clé primaire. Tenter de fixer stock à -1 viole le CHECK sur stock. La base refuse les données invalides au lieu de les stocker en espérant qu’une requête ultérieure les remarque. Lisez le nom de la contrainte et le message d’erreur. Un échec ne demande pas de supprimer la règle ; corrigez d’abord les données ou l’opération.

Résultat / données d’exemple

Tentative	Résultat attendu	Raison
Insérer de nouveau product_id 900	Erreur de contrainte d’unicité	L’identifiant existe déjà
Insérer un name NULL	Erreur NOT NULL	Un nom est requis
Fixer stock à -1	Erreur CHECK	Le stock négatif est interdit
Utiliser un nouvel ID et des valeurs valides	Accepté	Toutes les contraintes sont respectées

Gardez les démonstrations d’erreurs attendues séparées d’un script qui doit se terminer sans erreur.

UPDATE modifie les lignes correspondantes avec SET

UPDATE choisit une table cible. SET donne la nouvelle valeur d’une colonne. WHERE identifie les lignes modifiées. Un UPDATE sans WHERE peut modifier toutes les lignes ; vérifiez donc d’abord la condition avec SELECT. RETURNING permet d’examiner les lignes concernées. Cet exemple change le prix d’un produit d’exercice ; il ne change pas le catalogue initial.

SQL / lire l’instruction une clause à la fois

```sql
UPDATE work.product_proposal
SET unit_price = 13.00
WHERE product_id = 900
RETURNING product_id, unit_price;
```

Résultat / données d’exemple

product_id	unit_price
900	13.00

Sens des mots-clés

UPDATE — Choisir la table à modifier.

SET — Attribuer la nouvelle valeur de colonne.

WHERE — Limiter les lignes modifiées.

Lignes concernées — Lignes effectivement trouvées et modifiées.

Une transaction permet d’examiner et d’annuler les modifications d’exercice

Une transaction regroupe le travail SQL en une unité. BEGIN la démarre, COMMIT la rend permanente et ROLLBACK abandonne ses modifications transactionnelles. Pour vous exercer sans risque, mettez à jour la proposition dans une transaction, examinez le résultat, puis annulez-la. SELECT dans votre transaction voit votre propre modification. Après ROLLBACK, le prix précédent revient. Le jour 4 expliquera ce que les autres connexions peuvent voir.

EXÉCUTER COMME LA TRANSACTION INDIQUÉE

```
BEGIN;
UPDATE work.product_proposal
SET unit_price = unit_price * 1.05
WHERE product_id = 900
RETURNING unit_price;
ROLLBACK;
SELECT unit_price FROM work.product_proposal
WHERE product_id = 900;
```

Résultat / données d’exemple

Moment	unit_price
Avant BEGIN	13.00
Après UPDATE dans cette transaction	13.65
Après ROLLBACK	13.00

Une transaction permet d’examiner et d’annuler les modifications d’exercice / guide des mots-clés

Utilisez ce vocabulaire pour lire l’exemple précédent. Chaque définition explique le rôle exact d’un mot ou symbole dans l’instruction. Relisez le SQL depuis sa table d’entrée jusqu’à ses conditions et sa sortie, en utilisant ces définitions pour expliquer ce qui est demandé au serveur.

Sens des mots-clés

BEGIN — Démarrer une transaction.

COMMIT — Conserver les modifications de la transaction.

ROLLBACK — Annuler ses modifications transactionnelles.

DELETE, TRUNCATE et DROP suppriment des choses différentes

DELETE supprime les lignes sélectionnées par une condition. TRUNCATE vide une table entière avec une opération et un comportement de verrouillage différents. DROP TABLE supprime la définition de la table ainsi que ses données. Ces noms ne sont pas interchangeables pour une même tâche. Utilisez le schéma jetable work pour les démonstrations ; notre exercice habituel est un DELETE ciblé qui peut être annulé.

EXÉCUTER COMME LA TRANSACTION INDIQUÉE

```
BEGIN;
DELETE FROM work.product_proposal
WHERE product_id = 900
RETURNING product_id;
ROLLBACK;
```

Résultat / données d’exemple

Opération	Reste-t-il des lignes ?	La définition reste-t-elle ?
DELETE ... WHERE	Lignes non appariées gardées	oui
TRUNCATE TABLE	Aucune ligne ne reste	oui
DROP TABLE	La table est supprimée	non

ALTER TABLE fait évoluer une définition

ALTER TABLE modifie une définition de table existante. ADD COLUMN introduit une propriété ; DROP COLUMN supprime une propriété et ses valeurs enregistrées. Cette démonstration ajoute une description facultative dans une transaction, puis l’annule. Une migration est une modification de schéma contrôlée et versionnée. Dans les applications réelles, les migrations doivent aussi prendre en compte les données existantes, les requêtes dépendantes et l’utilisation concurrente.

EXÉCUTER COMME LA TRANSACTION INDIQUÉE

```
BEGIN;
ALTER TABLE work.product_proposal
ADD COLUMN description text;
ROLLBACK;
```

Sens des mots-clés

ALTER TABLE — Modifier une définition de table existante.

ADD COLUMN — Introduire une nouvelle propriété nommée.

Migration — Étape enregistrée de modification du schéma.

Les faits répétés causent des anomalies de mise à jour

Supposons que chaque ligne d’étudiant répète un nom de département. Si Computing est renommé, toutes les copies doivent changer de façon cohérente. En oublier une crée des faits contradictoires. C’est une anomalie de mise à jour : la structure exige plusieurs modifications coordonnées pour un seul changement réel. Stockez le fait du département une fois, attribuez-lui un identifiant et faites référencer cet identifiant par les étudiants.

Informations de département répétées

student	department_id	department_name
Alice	1	Computing
Bilal	1	Computing

Une ligne de département faisant autorité

department_id	department_name
1	Computing

La normalisation organise les faits pour réduire les anomalies liées aux dépendances.

Les dépendances fonctionnelles expliquent la séparation des tables

Une dépendance fonctionnelle « X détermine Y » signifie que lorsque des lignes ont le même X, elles doivent avoir le même Y. Un identifiant de produit détermine son nom. Un identifiant de commande détermine son étudiant. Mais un identifiant de commande seul ne détermine pas la quantité de chaque produit : la ligne nécessite order_id et product_id. Ces règles décrivent le sens des données, pas un motif fortuit dans un échantillon.

Résultat / données d’exemple

Dépendance	Interprétation
product_id -> product_name	Une identité de produit a un seul nom actuel
order_id -> student_id	Une commande appartient à un étudiant
(order_id, product_id) -> quantity	Une clé de ligne détermine sa quantité

Un déterminant est le membre gauche d’une dépendance fonctionnelle.

Les formes normales sont des contrôles de conception de plus en plus stricts

Les formes normales aident à vérifier que les faits dépendent des bonnes clés. La première forme normale traite chaque ligne et attribut comme une valeur relationnelle bien définie. La deuxième élimine les dépendances envers une seule partie d’une clé composée. La troisième traite les dépendances via des attributs non clés. BCNF est plus stricte : tout déterminant non trivial doit être une superclé, un ensemble de colonnes identifiant une ligne de façon unique.

Résultat / données d’exemple

Vérification	Question à poser	Exemple de correction
1NF	Chaque champ représente-t-il une valeur de son domaine prévu ?	Séparer une liste répétée de lignes de commande
2NF	Un fait non clé dépend-il de la clé composée entière ?	Déplacer product_name hors de order_items
3NF	Un fait non clé dépend-il d’un autre fait non clé ?	Déplacer department_name vers departments
BCNF	Tout déterminant non trivial est-il une superclé ?	Analyser toutes les clés candidates et dépendances

Apprenez à expliquer la dépendance et l’anomalie ; mémoriser les numéros des formes ne suffit pas.

À vous de jouer : identifier les faits mal placés

Une ébauche de table order_lines contient order_id, product_id, student_email, product_name et quantity. Une commande peut comporter plusieurs produits. Expliquez le sens prévu d’une ligne et identifiez les faits à stocker ailleurs. Cet exercice ne nécessite pas de SQL. Utilisez les dépendances de la diapositive précédente et expliquez une anomalie de mise à jour causée par cette structure.

Suivez le raisonnement

1. Indiquez quelles colonnes identifient une ligne.
2. Demandez-vous ce qui détermine student_email et product_name.
3. Dessinez des tables séparées reliées par des identifiants.

Corrigé expliqué : garder les faits avec leurs propriétaires

Une ligne est identifiée par la paire order_id et product_id. La quantité appartient à cette ligne. Le nom du produit appartient à products, car product_id le détermine. L’adresse électronique de l’étudiant appartient à students ; une commande référence son étudiant au lieu de recopier ses coordonnées dans chaque ligne. Cela sépare les informations d’identité actuelles des faits d’une commande.

Résultat / données d’exemple

Table	Colonnes importantes	Sens d’une ligne
students	student_id, email	Un étudiant
orders	order_id, student_id	Une commande
products	product_id, name	Un produit
order_items	order_id, product_id, quantity	Une ligne de produit dans une commande

Le prix de vente appartient toujours à la ligne de commande, car il enregistre le prix historique de la transaction.

Un agrégat résume plusieurs lignes en une valeur

Une fonction d’agrégation combine des valeurs d’entrée en une synthèse. COUNT compte les lignes ou valeurs présentes, SUM additionne les valeurs numériques, AVG calcule leur moyenne et MIN et MAX trouvent les extrêmes. Sans GROUP BY, la requête ci-dessous produit une ligne de synthèse pour toute la table products. Elle répond à une question de stock, pas de ventes.

SQL / lire l’instruction une clause à la fois

```sql
SELECT COUNT(*) AS product_count,
       SUM(stock) AS units_in_stock,
       MIN(unit_price) AS lowest_price
FROM campus.products;
```

Résultat / données d’exemple

product_count	units_in_stock	lowest_price
6	250	5.00

Sens des mots-clés

COUNT(*) — Compter chaque ligne d’entrée.

SUM(column) — Additionner les valeurs numériques présentes.

AVG / MIN / MAX — Moyenne / plus petite / plus grande valeur présente.

COUNT(*) et COUNT(column) traitent NULL différemment

COUNT(*) compte les lignes, que leurs colonnes soient présentes ou manquantes. COUNT(column) compte uniquement les lignes dont cette colonne n’est pas NULL. La plupart des agrégats ignorent les entrées NULL. categories contient cinq lignes mais seulement quatre identifiants de parent connus, car la racine n’a pas de parent. SUM sans ligne d’entrée renvoie NULL ; COUNT sans ligne renvoie zéro.

SQL / lire l’instruction une clause à la fois

```
SELECT COUNT(*) AS categories,
       COUNT(parent_id) AS known_parents
FROM campus.categories;
```

Résultat / données d’exemple

categories	known_parents
5	4

Sens des mots-clés

COUNT(DISTINCT column) — Compter les valeurs non NULL différentes.

Entrée vide — COUNT renvoie 0 ; SUM renvoie généralement NULL.

GROUP BY crée une ligne de synthèse par groupe

GROUP BY répartit les lignes en groupes partageant les mêmes valeurs de regroupement. Les fonctions d’agrégation calculent ensuite séparément dans chaque groupe. Regrouper par category_id fait passer le résultat d’une ligne par produit à une ligne par catégorie représentée par des produits. Toute expression sélectionnée non agrégée doit normalement être une expression de regroupement ou être autorisée par une règle applicable de dépendance fonctionnelle.

SQL / lire l’instruction une clause à la fois

```sql
SELECT category_id,
       COUNT(*) AS products,
       SUM(stock) AS units
FROM campus.products
GROUP BY category_id
ORDER BY category_id;
```

Sens des mots-clés

GROUP BY — Définir les groupes à synthétiser.

Clé de regroupement — La ou les valeurs identifiant un groupe.

Nouvelle granularité du résultat — Ici, une ligne par category_id.

Lire un résultat regroupé un groupe à la fois

La catégorie 4 contient Mouse et USB Hub. Leurs stocks valent 30 et 10 ; ce groupe compte donc deux produits et 40 unités. La catégorie 5 contient Notebook et Sticker Pack, soit deux produits et 150 unités. Le résultat n’affiche pas les noms individuels des produits, car ces lignes ont été combinées en synthèses par catégorie.

Résultat / données d’exemple

category_id	products	units
2	1	20
3	1	40
4	2	40
5	2	150

Vérifiez une synthèse en calculant manuellement un petit groupe avant de vous fier au résultat complet.

WHERE filtre les lignes ; HAVING filtre les groupes

WHERE décide quelles lignes individuelles atteignent le regroupement. HAVING décide quels groupes terminés restent. La requête conserve d’abord les produits actifs, les regroupe par catégorie, puis garde les catégories ayant au moins deux produits actifs. Après exclusion du produit inactif USB Hub, la catégorie 4 ne compte plus qu’un produit admissible. La catégorie 5 en compte toujours deux : c’est donc le seul groupe renvoyé.

SQL / lire l’instruction une clause à la fois

```sql
SELECT category_id, COUNT(*) AS active_products
FROM campus.products
WHERE active
GROUP BY category_id
HAVING COUNT(*) >= 2
ORDER BY category_id;
```

Résultat / données d’exemple

category_id	active_products
5	2

Sens des mots-clés

WHERE active — Filtrer les lignes de produits avant le regroupement.

HAVING COUNT(*) >= 2 — Filtrer les groupes après le calcul de leurs effectifs.

VALUES crée un petit ensemble de lignes littérales pour expérimenter

VALUES construit des lignes directement à partir de valeurs littérales. Il permet de tester une idée sans créer de table permanente ni modifier le jeu de données de référence du cours. Chaque groupe entre parenthèses est une ligne et les positions correspondent aux colonnes. PostgreSQL donne à ces colonnes anonymes des noms par défaut comme column1 et column2. Plus tard, les tables dérivées et CTE permettront de leur donner des noms pertinents.

SQL / lire l’instruction une clause à la fois

```sql
VALUES (1, 'Pen'),
       (2, 'Book');
```

Résultat / données d’exemple

column1	column2
1	Pen
2	Book

Sens des mots-clés

VALUES — Construire des lignes à partir des valeurs fournies.

(...) — Valeurs ordonnées d’une ligne.

Même structure de ligne — Chaque ligne fournit le même nombre de colonnes compatibles.

À vous de jouer : prouver un agrégat au lieu de deviner

Indiquez le nombre de produits actifs et leur stock total. Prévoyez d’abord la réponse en examinant le jeu de données de référence à six produits. Écrivez ensuite la requête d’agrégation. Il faut une condition WHERE et deux fonctions d’agrégation. N’ajoutez pas GROUP BY : la question demande une synthèse globale, pas une synthèse par catégorie.

Suivez le raisonnement

1. Excluez USB Hub, car active est faux.
2. Comptez les cinq produits restants.
3. Additionnez leurs stocks : 20 + 30 + 50 + 40 + 100.

Corrigé expliqué : une entrée filtrée, une ligne de synthèse

WHERE élimine le produit inactif avant l’agrégation. COUNT(*) compte donc cinq lignes. SUM(stock) additionne les cinq valeurs de stock restantes et produit 240. GROUP BY est inutile, car tous les produits admissibles contribuent à une seule synthèse. Si vous regroupez par erreur par product_id, vous obtiendrez une synthèse par produit et répondrez à une autre question.

SQL / lire l’instruction une clause à la fois

```sql
SELECT COUNT(*) AS active_products,
       SUM(stock) AS active_units
FROM campus.products
WHERE active;
```

Résultat / données d’exemple

active_products	active_units
5	240

Point de contrôle du jour 1 : expliquer avant d’ajouter de la syntaxe

Vous savez maintenant vous connecter à PostgreSQL, lire et filtrer une table, expliquer les valeurs manquantes, définir des tables avec contraintes, pratiquer des modifications réversibles et calculer des synthèses regroupées. Avant le TP01, expliquez chaque élément ci-dessous à un camarade avec un exemple concret. Demain, nous relierons les tables par des jointures ; le TP d’aujourd’hui évite d’exiger des jointures encore non enseignées.

Résultat / données d’exemple

Question	Mots que vous devez savoir utiliser
Que représente une ligne ?	Table, colonne, identifiant, sens d’une ligne
Comment sélectionner un résultat ?	SELECT, FROM, WHERE, ORDER BY
Pourquoi les données manquantes peuvent-elles se comporter différemment ?	NULL, inconnu, IS NULL
Comment garantir la validité des données ?	Type, clé, NOT NULL, CHECK, clé étrangère
Comment annuler une modification ?	BEGIN, RETURNING, ROLLBACK
Comment fonctionnent les synthèses ?	Agrégat, GROUP BY, différence entre WHERE et HAVING

Le TP01 construit ces bases. Le TP02 présente et fait pratiquer les jointures avant les rapports croisant plusieurs tables.